

CSS Layout & Positioning

Lesson Overview

In this lesson, we'll explore how to control the layout and positioning of elements on a webpage. By the end, you'll understand how to arrange elements precisely where you want them.

PART 1: CSS Display Property

Step 1: Understanding Display Values

The `display` property determines how an element behaves in the layout flow.

Basic Display Types:

Block Elements (`display: block`)

- Take up full width available
- Start on a new line
- Can set width, height, margin, padding
- Examples: `<div>`, `<p>`, `<h1>`-`<h6>`

```
.block-example {  
  
  display: block;  
  
  width: 300px;  
  
  height: 100px;  
  
  background: lightblue;  
  
}
```

Inline Elements (`display: inline`)

- Only take up as much width as needed
- Do NOT start on a new line
- Cannot set width or height (only padding and margin horizontally)
- Examples: `<span>`, `<a>`, `<strong>`

```
.inline-example {  
  
  display: inline;
```

```
background: lightgreen;

/* width and height won't work here! */

}
```

Inline-block Elements (`display: inline-block`)

- Mix of both: flows like inline BUT can set width/height
- Great for creating horizontal layouts

```
.inline-block-example {

display: inline-block;

width: 150px;

height: 50px;

background: lightcoral;

margin: 10px;

}
```

Step 2: Special Display Values

None (`display: none`)

- Completely removes element from the page
- Element is not visible and takes no space

```
.hidden-element {

display: none; /* completely gone */

}
```

Table & List-item (`display: table`, `display: list-item`)

- Makes elements behave like HTML table or list items
- Less commonly used but useful for specific layouts

```
.table-style {

display: table;

width: 100%;

}
```

```
}  
  
.cell-style {  
  
    display: table-cell;  
  
    padding: 10px;  
  
}
```

Step 3: Modern Layout Systems

Flexbox (`display: flex`)

- One-dimensional layout system
- Great for arranging items in rows or columns
- We'll cover this in detail in a future lesson

```
.flex-container {  
  
    display: flex;  
  
    justify-content: center; /* horizontal alignment */  
  
    align-items: center; /* vertical alignment */  
  
}
```

Grid (`display: grid`)

- Two-dimensional layout system
- Perfect for complex page layouts
- Also covered in a future lesson

```
.grid-container {  
  
    display: grid;  
  
    grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */  
  
    gap: 20px;  
  
}
```

CSS Positioning

Step 4: Static Positioning (Default)

Static (`position: static`)

- Default positioning for all elements
- Elements flow in normal document order
- Cannot use top, right, bottom, left properties

```
.static-element {  
  
    position: static; /* default - nothing special */  
  
}
```

Step 5: Relative Positioning

Relative (`position: relative`)

- Element stays in normal document flow
- Can be offset from its normal position using top/right/bottom/left
- Other elements behave as if it's still in its original position

```
.relative-example {  
  
    position: relative;  
  
    top: 20px;      /* moves down 20px from original position */  
  
    left: 30px;     /* moves right 30px from original position */  
  
    background: yellow;  
  
}
```

Key Point: When you move a relative element, other elements don't shift to fill the gap - it's like the element creates a "ghost" at its original position.

Step 6: Absolute Positioning

Absolute (`position: absolute`)

- Removed from normal document flow
- Positioned relative to the nearest positioned ancestor (non-static)
- If no positioned ancestor, positions relative to the `<html>` element

```
.absolute-example {  
  
    position: absolute;  
  
    top: 50px;  
  
    right: 20px;  
  
    background: orange;  
  
    width: 200px;  
  
    height: 100px;  
  
}
```

Important: Use absolute positioning with a relatively positioned parent container to create controlled layouts.

```
.parent-container {  
  
    position: relative; /* becomes the reference point */  
  
    height: 400px;  
  
    border: 2px solid black;  
  
}  
  
.child-absolute {  
  
    position: absolute;  
  
    bottom: 20px; /* 20px from bottom of parent */  
  
    right: 20px; /* 20px from right of parent */  
  
    background: purple;  
  
    color: white;  
  
    padding: 10px;  
  
}
```

Step 7: Fixed Positioning

Fixed (`position: fixed`)

- Removed from normal document flow
- Positioned relative to the viewport (browser window)
- Stays in the same position even when scrolling

```
.fixed-example {  
  
    position: fixed;  
  
    top: 0;  
  
    left: 0;  
  
    width: 100%;  
  
    background: navy;  
  
    color: white;  
  
    padding: 20px;  
  
    z-index: 1000; /* ensures it stays on top */  
  
}
```

Common Uses:

- Sticky headers
- Floating action buttons
- Navigation bars
- Cookie consent banners

Step 8: Sticky Positioning

Sticky (`position: sticky`)

- Hybrid of relative and fixed
- Behaves like relative until it reaches a scroll threshold
- Then "sticks" in place like fixed positioning

```
.sticky-example {  
  
    position: sticky;  
  
    top: 0;  
  
    background: lightgray;  
  
    padding: 15px;
```

```
z-index: 100;

}
```

Key Points:

- Must specify at least one threshold (top, bottom, left, or right)
- Stays within its parent container boundaries
- Great for section headers in long content

Z-index and Stacking Context

Step 9: Understanding Z-index

Z-index controls the stacking order of positioned elements.

```
.element1 {

  position: absolute;

  z-index: 1;

  background: red;

}

.element2 {

  position: absolute;

  z-index: 2;

  background: blue;

  /* Appears above element1 because z-index is higher */

}
```

Rules to Remember:

1. Only works on positioned elements (not static)
2. Higher numbers appear on top
3. Can use negative values
4. Default z-index is `auto` (which is 0)

Step 10: Stacking Context

A stacking context is a 3D conceptual space where elements stack.

Creating a Stacking Context:

- Root element (<html>)
- Elements with `position` set and `z-index` set
- Elements with `opacity` less than 1
- Elements with `transform` or `filter` properties

```
.parent {  
  
  position: relative;  
  
  z-index: 10; /* Creates a stacking context */  
  
}  
  
.child-high {  
  
  position: absolute;  
  
  z-index: 999; /* Only stacks within parent's context */  
  
}
```

Important: Children with high z-index can't escape their parent's stacking context.

ACTIVITY: Build a Profile Card

Objective

Create a modern profile card using display and positioning properties.

Instructions

Task 1: HTML Structure Create this HTML structure:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
  <title>Profile Card Activity</title>  
  
  <link rel="stylesheet" href="style.css">
```

```
</head>

<body>

  <!-- Navigation (fixed) -->

  <nav class="navbar">

    <span>My Portfolio</span>

    <div class="nav-links">

      <a href="#">Home</a>

      <a href="#">About</a>

      <a href="#">Contact</a>

    </div>

  </nav>

  <!-- Main Profile Card -->

  <div class="card-container">

    <div class="profile-card">

      <!-- Badge (absolute positioned) -->

      <div class="badge">PRO</div>

      <!-- Profile Image -->

      <div class="profile-image">

        

      </div>

      <!-- Profile Info -->

      <h2 class="name">Jane Doe</h2>

    </div>

  </div>

</body>

</html>
```

```
<p class="title">Senior Web Developer</p>
```

```
<p class="bio">Passionate about creating beautiful, responsive web experiences.</p>
```

```
<!-- Social Icons -->
```

```
<div class="social-icons">
```

```
<span class="icon">📱</span>
```

```
<span class="icon">🐦</span>
```

```
<span class="icon">👛</span>
```

```
<span class="icon">📧</span>
```

```
</div>
```

```
<!-- Follow Button (sticky within card) -->
```

```
<button class="follow-btn">Follow +</button>
```

```
</div>
```

```
<!-- Sticky Section Header -->
```

```
<div class="section-header">
```

```
<h3>Recent Activity</h3>
```

```
</div>
```

```
<!-- Content -->
```

```
<div class="content">
```

```
<p>Scroll to see sticky behavior...</p>
```

```
<!-- Add some filler content -->
```

```
<div style="height: 500px;"></div>

<p>More content here...</p>

<div style="height: 500px;"></div>

<p>Keep scrolling!</p>

</div>

</div>

</body>

</html>
```

Task 2: CSS Implementation

Create `style.css` and implement the following:

```
/* Reset and base styles */

* {

  margin: 0;

  padding: 0;

  box-sizing: border-box;

}

body {

  font-family: Arial, sans-serif;

  background: #f0f2f5;

  min-height: 2000px; /* For scrolling demonstration */

}

/* 1. FIXED NAVIGATION */

.navbar {

  /* Make it fixed at the top */
```

```
/* Set width to 100% */

/* Add background color and padding */

/* Add a bottom shadow */

/* Set z-index to ensure it's on top */
}

.nav-links {

/* Use inline-block to display links horizontally */
}

.nav-links a {

/* Display as inline-block */

/* Add padding and text decoration */
}

/* 2. CARD CONTAINER */

.card-container {

/* Center the container */

max-width: 500px;

margin: 80px auto 20px;

padding: 0 20px;
}

/* 3. PROFILE CARD */

.profile-card {

/* Use relative positioning (needed for absolute badge) */

/* Add background color, border-radius, padding */

/* Add box-shadow for depth */
```

```
    position: relative;

    text-align: center;
}

/* 4. ABSOLUTE BADGE */

.badge {

    /* Position absolutely in top-right corner of card */

    /* Add background color, text color */

    /* Add padding, border-radius */

    /* Make it small and rounded */

}

/* 5. PROFILE IMAGE */

.profile-image {

    /* Center the image */

    /* Consider using display: inline-block */

}

.profile-image img {

    /* Make it round (border-radius: 50%) */

    /* Set width and height */

    /* Add border */

}

/* 6. NAME AND TITLE */

.name {

    /* Add margin and color */

}
```

```
.title {

    /* Style differently from name */

    /* Perhaps different color and size */

}

/* 7. SOCIAL ICONS */

.social-icons {

    /* Use flex or inline-block to display icons in a row */

    /* Center them */

    margin: 20px 0;

}

.icon {

    /* Display as inline-block */

    /* Add padding and background */

    /* Make them round */

    font-size: 20px;

}

/* 8. STICKY FOLLOW BUTTON */

.follow-btn {

    /* Make it sticky, stick to bottom of card */

    /* But we want it to stick when scrolling past card */

    /* Use position: sticky with bottom property */

    /* Note: sticky works within parent container */

    position: sticky;

    bottom: 10px;
```

```
/* Add background color, text color, padding, border-radius */

/* Make it full width */

}

/* 9. STICKY SECTION HEADER */

.section-header {

/* Make it sticky at top (when scrolled) */

/* Add background color and padding */

/* Add margin at top */

position: sticky;

top: 70px; /* Below the fixed navbar */

/* Add z-index if needed */

}

/* 10. ADD SOME STYLING TO CONTENT */

.content {

/* Add padding and background */

margin-top: 20px;

}
```

Challenge Questions

1. Why does the badge use absolute positioning?

- It needs to be positioned precisely in the corner of the card regardless of other content.

2. Why does the navbar use fixed positioning?

- It should stay visible at the top of the viewport when scrolling.

3. What happens to the follow button when you scroll?

- With sticky positioning, it stays at the bottom of the card until the card scrolls past.

4. Why does the section header use sticky positioning?

- It sticks below the navbar when scrolling through content, but stays within its container.

Bonus Challenge

Add a "back to top" button that:

- Uses fixed positioning
- Appears only when scrolled down (using JavaScript or CSS)
- Has a higher z-index than other elements

Reflection Questions

1. When would you use `display: inline-block` instead of `display: inline`?
2. What's the difference between `position: relative` and `position: absolute`?
3. How does `position: sticky` differ from `position: fixed`?
4. Why does `z-index` sometimes not work as expected?

Summary

Property	Values	Key Use
Display	block, inline, inline-block	Controls element flow
	none, flex, grid	Hiding or modern layouts

Property	Values	Key Use
	table, list-item	Special behaviors
Position	static	Default document flow
	relative	Offset from normal position
	absolute	Relative to nearest positioned parent
	fixed	Relative to viewport
	sticky	Hybrid relative/fixed
Z-index	Number	Controls stacking order